2 Reinforcement Learning

Multi-agent reinforcement learning (MARL) is, in essence, reinforcement learning (RL) applied to multi-agent game models to learn optimal policies for the agents. Thus, MARL is deeply rooted in both RL theory and game theory. This chapter provides a basic introduction to the theory and algorithms of RL when there is only a single agent for which we want to learn an optimal policy. We will begin by providing a general definition of RL, following which we will introduce the Markov decision process (MDP) as the foundational model used in RL to represent single-agent decision processes. Based on the MDP model, we will define basic concepts such as expected returns, optimal policies, value functions, and Bellman equations. The goal in an RL problem is to learn an optimal policy that chooses actions to achieve some objective, such as maximizing the expected (discounted) return in each state of the environment (Figure 2.1).

We will then introduce two basic families of algorithms to compute optimal policies for MDPs: dynamic programming and temporal-difference learning. Dynamic programming requires complete knowledge of the MDP specification and uses this knowledge to compute optimal value functions and policies. In contrast, temporal-difference learning does not require complete knowledge of the MDP; instead, it learns optimal value functions and policies by interacting with the environment and generating experiences. Most of the MARL algorithms introduced in Chapter 6 build on these families of algorithms and essentially extend them to game models.

Part I of this book focuses on the basic models and concepts used in MARL, and as such this chapter focuses only on the basic RL concepts that are required to understand the following chapters of this part of the book. In particular, topics such as value and policy function approximation are not covered in this chapter. These latter topics will be covered in Chapters 7 and 8 in Part II of the book.

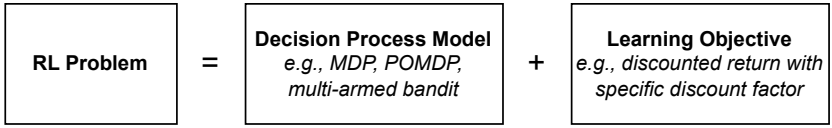


Figure 2.1: An RL problem is defined by the combination of a decision process model that defines the mechanics of the agent-environment interaction and a learning objective that specifies the properties of the optimal policy to be learned (e.g., maximize expected discounted return in each state).

2.1 General Definition

We begin by providing a general definition of reinforcement learning:

Reinforcement learning (RL) algorithms learn solutions for sequential decision processes via repeated interaction with an environment.

This definition raises three main questions:

- What is a sequential decision process?
- What is a solution to the process?
- What is learning via repeated interaction?

A sequential decision process is defined by an agent that makes decisions over multiple time steps within an environment to achieve a specified goal. In each time step, the agent receives an observation from the environment and chooses an action. In a fully observable setting, as we assume in this chapter, the agent observes the full state of the environment; but in general, observations may be incomplete and noisy. Given the chosen action, the environment may change its state according to some transition dynamics and send a scalar reward signal to the agent. Figure 2.2 summarizes this process.

A solution to the decision process is an optimal decision policy for the agent, which chooses actions in each state to achieve some specified learning objective. Typically, the learning objective is to maximize the expected return for the agent in each possible state.[1] The return in a state when following a given policy is defined as the sum of rewards received over time from that state onward. Thus, RL assumes that the goal in the decision process can, in principle, be framed as the maximization of expected returns.

1. Other learning objectives can be specified, such as maximizing the average reward (Sutton and Barto 2018) and different types of "risk-sensitive" objectives (Mihatsch and Neuneier 2002).

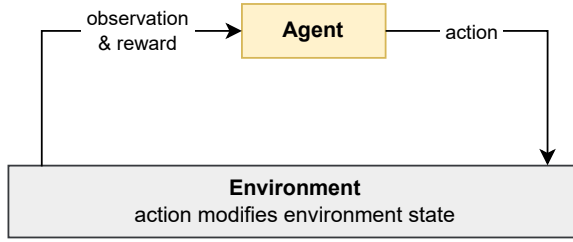


Figure 2.2: Basic reinforcement learning loop for a single-agent system.

Finally, RL algorithms learn such optimal policies by trying different actions in different states and observing the outcomes. This way of learning is sometimes described as "trial and error" since the actions may lead to positive or negative outcomes that are not known beforehand and, therefore, must be discovered by trying the actions. A central problem in this learning process, often called the *exploration-exploitation dilemma*, is how to balance exploring the outcomes of different actions versus sticking with actions that are currently believed to be best. Exploration may discover better actions but can accrue low rewards in the process, while exploitation achieves a certain level of returns but may not discover the optimal actions.

RL is a type of machine learning that differs from other types such as supervised learning and unsupervised learning. In supervised learning, we have access to a set of labeled input-output pairs $\{x_i, y_i\}$ of some unknown function $f(x_i) = y_i$, and the goal is to learn this function using the data. In unsupervised learning, we have access to some unlabeled data $\{x_i\}$ and the goal is to identify some useful structure within the data. RL is neither of these: RL is not supervised learning because the reward signals do not tell the agent which action to take in each state x_i, and thus do not act as a supervision signal y_i. This is because some actions may give lower immediate reward but may lead to states from which the agent can eventually receive higher rewards. RL also differs from unsupervised learning because the rewards, while not a supervised signal, act as a proxy from which to learn an optimal policy.

In the following sections, we will formally define these concepts — sequential decision processes, optimal policies, and learning by interaction — within a framework called the Markov decision process.

2.2 Markov Decision Processes

The standard model used in RL to define the sequential decision process is the *Markov decision process*:

Definition 1 (Markov decision process) *A finite Markov decision process (MDP) consists of:*

- *Finite set of states S, with subset of terminal states* $\bar{S} \subset S$
- *Finite set of actions A*
- *Reward function* $\mathcal{R} : S \times A \times S \to \mathbb{R}$
- *State transition probability function* $\mathcal{T} : S \times A \times S \to [0, 1]$ *such that*

$$\forall s \in S, a \in A : \sum_{s' \in S} \mathcal{T}(s, a, s') = 1 \tag{2.1}$$

- *Initial state distribution* $\mu : S \to [0, 1]$ *such that*

$$\sum_{s \in S} \mu(s) = 1 \quad \textit{and} \quad \forall s \in \bar{S} : \mu(s) = 0 \tag{2.2}$$

An MDP starts in an initial state $s^0 \in S$, which is sampled from μ. At time t, the agent observes the current state $s^t \in S$ of the MDP and chooses an action $a^t \in A$ with probability given by its policy, $\pi(a^t \mid s^t)$, which is conditioned on the state. Given the state s^t and action a^t, the MDP transitions into a next state $s^{t+1} \in S$ with probability given by $\mathcal{T}(s^t, a^t, s^{t+1})$, and the agent receives a reward $r^t = \mathcal{R}(s^t, a^t, s^{t+1})$. We also write this probability as $\mathcal{T}(s^{t+1} \mid s^t, a^t)$ to emphasize that it is conditioned on the state-action pair s^t, a^t. These steps are repeated until the process reaches a terminal state $s^t \in \bar{S}$ or after completing a maximum number of T time steps,[2] after which the process terminates; or the steps may be repeated for an infinite number of time steps if the MDP is non-terminating. Each independent run of this process is referred to as an *episode*.

A finite MDP can be compactly represented as a finite state machine. Figure 2.3 shows an example MDP, in which a Mars rover has collected some samples and must return to the base station. From the *Start* state, there are two paths that lead to the *Base* target state. The rover can travel down a steep mountain slope (action *right*) that will take it directly to the base station. However, there is a high probability of 0.5 that the rover will fall down the cliff and be destroyed (-10 reward). Or the rover can travel through a longer path (action *left*, then two times action *right*), which passes by two sites (*Site A* and *Site B*)

2. Technically, if we assume that the agent can observe the time step t, then termination after T time steps can be modeled by including t in the information contained in the state s^t and defining the set of terminal states $\bar{S}$ to contain all states s^t with $t \geq T$.

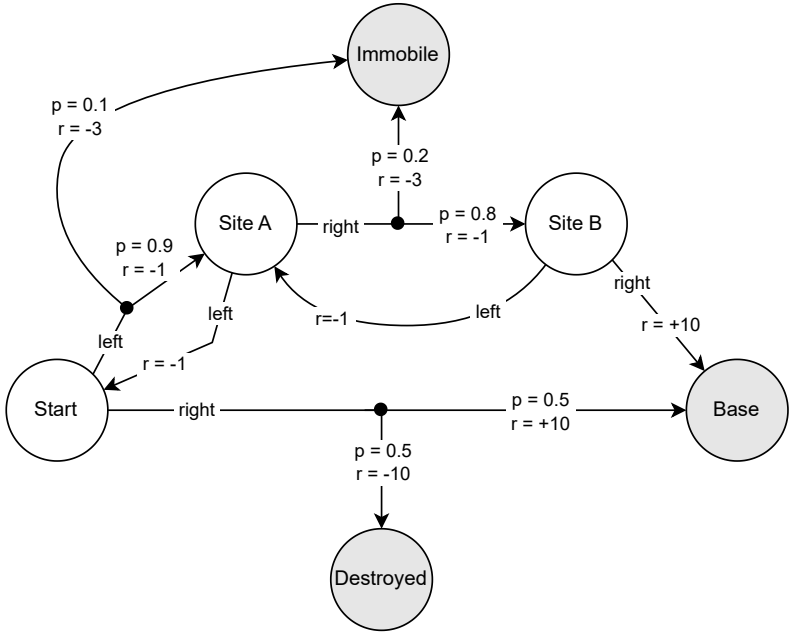


Figure 2.3: Mars Rover MDP. States are shown as circles, where *Start* is the initial state. Each non-terminal state (white) has two possible actions, *right* and *left*, that are shown as directed edges with associated transition probabilities (p) and rewards (r). Gray shaded circles mark terminal states.

before arriving at the base station. However, it takes a day to arrive at each site (-1 reward), and there is a probability of 0.3 that the rover will get stuck on the rocky ground and be immobilized (-3 reward). Arriving at the base station gives a reward of +10.

The term "Markov" comes from the *Markov property*, which states that the future state and reward are conditionally independent of past states and actions, given the current state and action:

$$\Pr(s^{t+1}, r^t \mid s^t, a^t, s^{t-1}, a^{t-1}, ..., s^0, a^0) = \Pr(s^{t+1}, r^t \mid s^t, a^t) \tag{2.3}$$

This means that the current state provides sufficient information to choose optimal actions in an MDP — past states and actions are not relevant.

The most common assumption in RL is that the dynamics of the MDP, in particular the transition and reward functions $\mathcal{T}$ and $\mathcal{R}$, are a priori unknown to the agent. Typically, the only parts of the MDP that are assumed to be known are the action space A and the state space S.

While Definition 1 defines MDPs with finite, or *discrete*, states and actions, MDPs can also be defined with continuous states and actions, or a mixture of both discrete and continuous elements. Moreover, while the reward function $\mathcal{R}$ as defined here is deterministic, MDPs can also define a probabilistic reward function such that $\mathcal{R}(s, a, s')$ gives a distribution over possible rewards.

The *multi-armed bandit problem* (or simply "bandit problem") is an important special case of the MDP in which each episode terminates after one time step (i.e., $T = 1$), there is only a single state in S and no terminal states (i.e., $|S| = 1$ and $\bar{S} = \emptyset$), and the reward function $\mathcal{R}$ is probabilistic and unknown to the agent. Thus, in such a bandit problem, each action produces a reward from some unknown reward distribution, and the goal is effectively to find the action that yields the highest expected reward as quickly as possible. Bandit problems have been used as a basic model to study the exploration-exploitation dilemma (Lattimore and Szepesvári 2020).

An inherent assumption in MDPs is that the agent can fully observe the state of the environment in each time step. In many applications, an agent only observes partial and noisy information about the environment. *Partially observable Markov decision processes* (POMDPs) (Kaelbling, Littman, and Cassandra 1998) generalize MDPs by defining a decision process in which the agent receives observations o^t rather than directly observing the state s^t, and these observations depend in a probabilistic or deterministic way on the state. Thus, in general, the agent will need to take into account the history of past observations $o^0, o^1, ..., o^t$ in order to infer the possible current states s^t of the environment. POMDPs are a special case of the partially observable stochastic game (POSG) model introduced in Chapter 3; namely, a POMDP is a POSG in which there is only one agent. We refer to Section 3.4 for a more detailed definition and discussion of partial observability in decision processes.

We have defined the MDP as a model of decision processes, but we have not yet defined the learning objective in an MDP. The next section will introduce the most common learning objective used in MDPs: maximizing expected discounted returns. As discussed earlier (Figure 2.1), together, an MDP and learning objective specify an RL problem.

2.3 Expected Discounted Returns and Optimal Policies

Given a policy π that specifies action probabilities in each state, and assuming that each episode in the MDP terminates after T time steps, the total *return* in an episode is the cumulative reward received over time

$$r^0 + r^1 + ... + r^{T-1}. \tag{2.4}$$

Due to the stochastic nature of the MDP, it may not be possible to maximize this return in all episodes. This is because some actions may lead to different outcomes with certain probabilities, and these outcomes are outside the control of the agent. Therefore, the agent can maximize the *expected return* given by

$$\mathbb{E}_\pi\left[r^0 + r^1 + ... + r^{T-1}\right] \tag{2.5}$$

where the expectation $\mathbb{E}_\pi$ assumes that the initial state is sampled from the initial state distribution (i.e., $s^0 \sim \mu$), that the agent follows policy π to select actions (i.e., $a^t \sim \pi(\cdot \mid s^t)$), and that successor states are governed by the state transition probabilities (i.e., $s^{t+1} \sim \mathcal{T}(\cdot \mid s^t, a^t)$).

The above definition of total returns is guaranteed to be finite for a terminating MDP. However, in non-terminating MDPs the total return may be infinite, in which case returns may not be informative to distinguish the performance of different policies that achieve infinite returns. The standard approach to ensuring finite returns in non-terminating MDPs is to use a *discount factor* $\gamma \in [0, 1]$, based on which we define the *discounted return*[3]

$$\mathbb{E}_\pi\left[r^0 + \gamma r^1 + \gamma^2 r^2 + ...\right] \;=\; \mathbb{E}_\pi\left[\sum_{t=0}^{\infty} \gamma^t r^t\right]. \tag{2.6}$$

For $\gamma < 1$ and assuming that rewards are constrained to lie in a finite range $[r_{\min}, r_{\max}]$, the discounted return is guaranteed to be finite

$$\sum_{t=0}^{\infty} \gamma^t r^t \;\leq\; r_{\max} \sum_{t=0}^{\infty} \gamma^t \;=\; r_{\max} \frac{1}{1-\gamma} \tag{2.7}$$

where the right-hand fraction in the above equation is the closed form of the geometric series given by $\sum_{t=0}^{\infty} \gamma^t$.

The discount factor has two equivalent interpretations. One interpretation is that $(1-\gamma)$ is the probability with which the MDP terminates after each time step. Thus, the probability that the MDP terminates after a total of $T > 0$ time steps is $\gamma^{T-1}(1-\gamma)$, where γ^{T-1} is the probability of continuing (i.e., not terminating) in the first $T-1$ time steps, multiplied by the probability $(1-\gamma)$ of terminating in the following time step. For example, in the Mars Rover MDP shown in Figure 2.3, we can specify a discount factor of $\gamma = 0.95$ to model the fact that the rover's battery may fail with a probability of 0.05 after each state transition. The second interpretation is that the agent gives "weight" γ^t to reward r^t received at time t. Thus, a γ close to 0 leads to a myopic agent that cares more about near-term rewards, while a γ close to 1 leads to a farsighted agent that also values distant rewards. In either case, it is important to note that

3. Note that r^t refers to the reward at time t, while γ^t denotes the exponentiation operation (γ to the power of t). This is a standard notational convention in RL literature.

the discount rate is part of the learning objective and not a tunable algorithm parameter; γ is a fixed parameter specified by the learning objective. In the remainder of this book, whenever we refer to returns, we specifically mean discounted returns with some discount factor γ.

Finally, the above definition of discounted returns can work for both terminating and non-terminating MDPs via the convention of *absorbing states*. If an episode reaches a terminal state or a maximum number of time steps, we define the final state in the episode to be absorbing in that any subsequent actions in this state will transition the MDP into the same state with probability 1 and give a reward of 0 to the agent. Thus, once the MDP reaches an absorbing state, it will forever remain in that state and the agent will not accrue any more rewards.

Based on the above definition of discounted returns and absorbing states, we can now define the solution to the MDP as the *optimal policy* π^* that maximizes the expected discounted return.

2.4 Value Functions and Bellman Equation

Based on the Markov property defined in Equation 2.3, we know that given the current state and action, the future states and rewards are independent of the past states and actions. This implies independence of past rewards, since rewards are a function of the states and actions, $r^t = \mathcal{R}(s^t, a^t, s^{t+1})$. Therefore, in an MDP, the expected return can be defined individually for each state $s \in S$. This gives rise to the concept of *value functions*, which are central to much of RL theory and many algorithms.

First, note that the discounted reward sequence can be written in recursive form as

$$u^t = r^t + \gamma r^{t+1} + \gamma^2 r^{t+2} + ... \tag{2.8}$$

$$= r^t + \gamma u^{t+1}. \tag{2.9}$$

Given a policy π, the *state-value function* $V^\pi(s)$ gives the "value" of state s under π, which is the expected return when starting in state s and following policy π to select actions, formally

$$V^\pi(s) = \mathbb{E}_\pi \left[u^t \mid s^t = s \right] \tag{2.10}$$

$$= \mathbb{E}_\pi \left[r^t + \gamma u^{t+1} \mid s^t = s \right] \tag{2.11}$$

$$= \sum_{a \in A} \pi(a \mid s) \sum_{s' \in S} \mathcal{T}(s' \mid s, a) \left[\mathcal{R}(s, a, s') + \gamma \mathbb{E}_\pi \left[u^{t+1} \mid s^{t+1} = s' \right] \right] \tag{2.12}$$

$$= \sum_{a \in A} \pi(a \mid s) \sum_{s' \in S} \mathcal{T}(s' \mid s, a) \left[\mathcal{R}(s, a, s') + \gamma V^\pi(s') \right] \tag{2.13}$$

where $V^\pi(s) = 0$ for terminal (i.e., absorbing) states $s \in \bar{S}$. The recursive equation given in Equation 2.13 is called the *Bellman equation* in honor of Richard Bellman and his pioneering work (Bellman 1957).

The Bellman equation for V^π defines a system of m linear equations with m variables, where $m = |S|$ is the number of states in the finite MDP:

$$V^\pi(s_1) = \sum_{a \in A} \pi(a \mid s_1) \sum_{s' \in S} \mathcal{T}(s' \mid s_1, a) \left[\mathcal{R}(s_1, a, s') + \gamma V^\pi(s') \right] \tag{2.14}$$

$$V^\pi(s_2) = \sum_{a \in A} \pi(a \mid s_2) \sum_{s' \in S} \mathcal{T}(s' \mid s_2, a) \left[\mathcal{R}(s_2, a, s') + \gamma V^\pi(s') \right] \tag{2.15}$$

$$\vdots$$

$$V^\pi(s_m) = \sum_{a \in A} \pi(a \mid s_m) \sum_{s' \in S} \mathcal{T}(s' \mid s_m, a) \left[\mathcal{R}(s_m, a, s') + \gamma V^\pi(s') \right] \tag{2.16}$$

where $V^\pi(s_k)$ for $k = 1, ..., m$ are the variables in the equation system and $\pi(a \mid s_k)$, $\mathcal{T}(s' \mid s_k, a)$, $\mathcal{R}(s_k, a, s')$, and γ are constants. This equation system has a unique solution given by the value function V^π for policy π. If all elements of the MDP are known, then one can solve this equation system to obtain V^π using any method to solve linear equation systems (such as Gauss elimination).

Analogous to state-value functions, we can define *action-value functions* $Q^\pi(s, a)$ which give the expected return when selecting action a in state s and then following policy π to select actions subsequently,

$$Q^\pi(s, a) = \mathbb{E}_\pi \left[u^t \mid s^t = s, a^t = a \right] \tag{2.17}$$

$$= \mathbb{E}_\pi \left[r^t + \gamma u^{t+1} \mid s^t = s, a^t = a \right] \tag{2.18}$$

$$= \sum_{s' \in S} \mathcal{T}(s' \mid s, a) \left[\mathcal{R}(s, a, s') + \gamma V^\pi(s') \right] \tag{2.19}$$

$$= \sum_{s' \in S} \mathcal{T}(s' \mid s, a) \left[\mathcal{R}(s, a, s') + \gamma \sum_{a' \in A} \pi(a' \mid s') Q^\pi(s', a') \right] \tag{2.20}$$

where $Q^\pi(s, a) = 0$ for terminal states $s \in \bar{S}$. For the jump from Equation 2.19 to 2.20, notice in Equation 2.13 that the second sum (over states s') is an application of Equation 2.19, and hence it can be replaced by $Q^\pi(s', a')$. Equation 2.20 admits a system of linear equations that has a unique solution given by Q^π.

A policy π is optimal in the MDP if the policy's (state or action) value function is the *optimal value function* of the MDP, defined as

$$V^*(s) = \max_{\pi'} V^{\pi'}(s), \ \forall s \in S \tag{2.21}$$

$$Q^*(s, a) = \max_{\pi'} Q^{\pi'}(s, a), \ \forall s \in S, a \in A \tag{2.22}$$

We use π^* to denote any optimal policy with optimal value function V^* or Q^*.

Because of the Bellman equation, this means that for any optimal policy π^* we have

$$\forall \pi' \, \forall s : V^*(s) \geq V^{\pi'}(s). \tag{2.23}$$

Thus, maximizing the expected return in an MDP amounts to maximizing the expected return in each possible state $s \in S$.

In fact, we can write the optimal value functions without reference to the policy using the *Bellman optimality equations*:

$$V^*(s) = \max_{a \in A} \sum_{s' \in S} \mathcal{T}(s' \mid s, a) \left[\mathcal{R}(s, a, s') + \gamma V^*(s') \right] \tag{2.24}$$

$$Q^*(s, a) = \sum_{s' \in S} \mathcal{T}(s' \mid s, a) \left[\mathcal{R}(s, a, s') + \gamma \max_{a' \in A} Q^*(s', a') \right] \tag{2.25}$$

The Bellman optimality equations define a system of m non-linear equations, where m is again the number of states in the finite MDP. The non-linearity is due to the max-operator used in the equations. The unique solution to the system is the optimal value function V^*/Q^*.

Once we know the optimal action-value function Q^*, the optimal policy π^* is simply derived by choosing actions with maximum value (i.e., expected return) in each state,

$$\pi^*(s) = \arg\max_{a \in A} Q^*(s, a) \tag{2.26}$$

where we use the =-notation in Equation 2.26 as a convenient shorthand to denote the deterministic policy that assigns probability 1 to the arg max-action in state s. If multiple actions have the same maximum value under Q^*, then the optimal policy can assign arbitrary probabilities to these actions (the probabilities must sum to 1). Therefore, while the optimal value function of the MDP is always unique, there may be multiple[4] optimal policies that have the same optimal (unique) value function. However, as shown in Equation 2.26, there always exist deterministic optimal policies in MDPs.

In the next sections, we will present two algorithm families to compute optimal value functions and policies. The first, dynamic programming, uses the Bellman (optimality) equations in an iterative way to estimate value functions, and it requires knowledge of the complete MDP such as the reward function and state transition probabilities. The second, temporal-difference learning, does not require complete knowledge of the MDP and instead uses sampled experiences from interactions with the environment to update value estimates.

4. In fact, if multiple actions have maximum value, then there will be an *infinite* number of optimal policies since the set of possible probability assignments to these actions spans a continuous (infinite) space.

2.5 Dynamic Programming

Dynamic programming (DP)[5] is a family of algorithms to compute value functions and optimal policies in MDPs (Bellman 1957; Howard 1960). DP algorithms use the Bellman equations as operators to iteratively estimate the value function and optimal policy. Thus, DP algorithms require complete knowledge of the MDP model, including the reward function $\mathcal{R}$ and state transition probabilities $\mathcal{T}$. While DP algorithms do not "interact" with the environment as per our definition in Section 2.1, the basic concepts underlying DP are an important building block in RL theory, including temporal-difference learning presented in Section 2.6.

The basic DP approach is *policy iteration*, which alternates between two tasks:

- **Policy evaluation:** compute value function V^π for current policy π
- **Policy improvement:** improve current policy π with respect to V^π

Thus, policy iteration produces a sequence of policy and value function estimates, starting with some initial policy π^0 (e.g., uniform-random policy) and value function V^0 (e.g., zero vector):

$$\pi^0 \to V^{\pi^0} \to \pi^1 \to V^{\pi^1} \to \pi^2 \to ... \to V^* \to \pi^* \tag{2.27}$$

As we will show later in this section, this sequence converges to the optimal value function, V^*, and optimal policy, π^*, under *greedy policy improvements*.

We first consider the policy evaluation task. Recall that the Bellman equation for V^π defines a system of linear equations, which can be solved to obtain V^π. However, using Gauss elimination (the de-facto standard solution approach for linear equation systems) has time complexity $O(m^3)$, where m is the number of states of the MDP. A number of alternative methods for policy evaluation have been developed in the DP literature, which often operate in iterative ways to produce successive approximations of value functions (see Puterman (2014) and Sutton and Barto (2018) for overviews). *Iterative policy evaluation* is one such method that iteratively applies the Bellman equation for V^π to produce successive estimates of V^π. The algorithm first initializes a vector $V^0(s) = 0$ for all $s \in S$. It then repeatedly performs update sweeps for all states $s \in S$:

$$V^{k+1}(s) \leftarrow \sum_{a \in A} \pi(a \,|\, s) \sum_{s' \in S} \mathcal{T}(s' \,|\, s, a) \left[\mathcal{R}(s, a, s') + \gamma V^k(s') \right] \tag{2.28}$$

5. The word "programming" in DP refers to a mathematical optimization problem, analogous to how the term is used in linear programming and non-linear programming. Regarding the adjective "dynamic," Bellman (1957) states in his preface that it "indicates that we are interested in processes in which time plays a significant role, and in which the order of operations may be crucial."

This sequence $V^0, V^1, V^2, ...$ converges to the value function V^π. Hence, in practice, we may stop the updates once there are no more changes between V^k and V^{k+1} after performing an update sweep. Notice that Equation 2.28 updates the value estimate for a state s using value estimates for other states s'. This property is called *bootstrapping* and is a core property of many RL algorithms.

As an example, we apply iterative policy evaluation in the Mars Rover problem from Section 2.2 with $\gamma = 0.95$. First, consider a policy π which in the initial state *Start* chooses action *right* with probability 1. (We ignore the other states since they will never be visited under this policy.) After we run iterative policy evaluation to convergence, we obtain a value of $V^\pi(Start) = 0$. From the MDP specification in Figure 2.3, it can be easily verified that this value is correct, since $V^\pi(Start) = -10 \cdot 0.5 + 10 \cdot 0.5 = 0$. Now, consider a policy π, which chooses both actions with equal probability 0.5 in the initial state and action *right* with probability 1 in states *Site A* and *Site B*. In this case, the converged values are $V^\pi(Start) = 2.05$, $V^\pi(Site\ A) = 6.2$, and $V^\pi(Site\ B) = 10$. For both policies, the values for the terminal states are all 0, since these are absorbing states as defined in Section 2.3.

To see why iterative policy evaluation converges to the value function V^π, one can show that the Bellman operator defined in Equation 2.28 is a *contraction mapping*. A mapping $f : \mathcal{X} \to \mathcal{X}$ on a $\|\cdot\|$–normed complete vector space $\mathcal{X}$ is a γ-contraction, for $\gamma \in [0, 1)$, if for all $x, y \in \mathcal{X}$:

$$\|f(x) - f(y)\| \leq \gamma \|x - y\| \tag{2.29}$$

By the Banach fixed-point theorem, if f is a contraction mapping, then for any initial vector $x \in \mathcal{X}$ the sequence $f(x), f(f(x)), f(f(f(x))), ...$ converges to a unique fixed point $x^* \in \mathcal{X}$ such that $f(x^*) = x^*$. Using the max-norm $\|x\|_\infty = \max_i |x_i|$, it can be shown that the Bellman equation indeed satisfies Equation 2.29. First, rewrite the Bellman equation as follows:

$$V^\pi(s) = \sum_{a \in A} \pi(a \mid s) \sum_{s' \in S} \mathcal{T}(s' \mid s, a) \left[\mathcal{R}(s, a, s') + \gamma V^\pi(s') \right] \tag{2.30}$$

$$= \sum_{a \in A} \sum_{s' \in S} \pi(a \mid s) \mathcal{T}(s' \mid s, a) \mathcal{R}(s, a, s') + \sum_{a \in A} \sum_{s' \in S} \pi(a \mid s) \mathcal{T}(s' \mid s, a) \gamma V^\pi(s') \tag{2.31}$$

This can be written as an operator $f^\pi(v)$ over a value vector $v \in \mathbb{R}^{|S|}$

$$f^\pi(v) = r^\pi + \gamma M^\pi v \tag{2.32}$$

where $r^\pi \in \mathbb{R}^{|S|}$ is a vector with elements

$$r_s^\pi = \sum_{a \in A} \sum_{s' \in S} \pi(a \mid s) \mathcal{T}(s' \mid s, a) \mathcal{R}(s, a, s') \tag{2.33}$$

and $M^\pi \in \mathbb{R}^{|S|\times|S|}$ is a matrix with elements

$$M^\pi_{s,s'} = \sum_{a\in A} \pi(a \mid s)\mathcal{T}(s' \mid s, a). \tag{2.34}$$

Then, we have for any two value vectors v, u:

$$\|f^\pi(v) - f^\pi(u)\|_\infty = \|(r^\pi + \gamma M^\pi v) - (r^\pi + \gamma M^\pi u)\|_\infty \tag{2.35}$$

$$= \gamma \|M^\pi (v - u)\|_\infty \tag{2.36}$$

$$\leq \gamma \|v - u\|_\infty. \tag{2.37}$$

Therefore, the Bellman operator is a γ-contraction under the max-norm, and repeated application of the Bellman operator converges to a unique fixed point, which is V^π. (The inequality in (2.37) holds because for each $s \in S$ we have $\sum_{s'} M^\pi_{s,s'} = 1$ and, therefore, $\|M^\pi x\|_\infty \leq \|x\|_\infty$ for any value vector x.)

Now that we have a method for policy evaluation, we next consider the policy improvement task in policy iteration. Once we have computed the value function V^π, the policy improvement task modifies the policy π by making it *greedy* with respect to V^π for all $s \in S$:

$$\pi' = \arg\max_{a\in A} \mathcal{T}(s' \mid s, a) \left[\mathcal{R}(s, a, s') + \gamma V^\pi(s')\right] \tag{2.38}$$

$$= \arg\max_{a\in A} Q^\pi(s, a) \tag{2.39}$$

By the *policy improvement theorem* (Sutton and Barto 2018), we know that if for all $s \in S$

$$\sum_{a\in A} \pi'(a \mid s) Q^\pi(s, a) \geq \sum_{a\in A} \pi(a \mid s) Q^\pi(s, a) \tag{2.40}$$

$$= V^\pi(s) \tag{2.41}$$

then π' must be as good as or better than π:

$$\forall s : V^{\pi'}(s) \geq V^\pi(s). \tag{2.42}$$

If after the policy improvement task, the greedy policy π' did not change from π, then we know that $V^{\pi'} = V^\pi$ and it follows for all $s \in S$:

$$V^{\pi'}(s) = \max_{a\in A} \mathbb{E}_\pi \left[r^t + \gamma V^\pi(s^{t+1}) \mid s^t = s, a^t = a\right] \tag{2.43}$$

$$= \max_{a\in A} \mathbb{E}_{\pi'} \left[r^t + \gamma V^{\pi'}(s^{t+1}) \mid s^t = s, a^t = a\right] \tag{2.44}$$

$$= \max_{a\in A} \sum_{s'\in S} \mathcal{T}(s' \mid s, a) \left[\mathcal{R}(s, a, s') + \gamma V^{\pi'}(s')\right] \tag{2.45}$$

$$= V^*(s) \tag{2.46}$$

Algorithm 1 Value iteration for MDPs

1: Initialize: $V(s) = 0$ for all $s \in S$

2: Repeat until V converged:

$$\forall s \in S: \; V(s) \leftarrow \max_{a \in A} \sum_{s' \in S} \mathcal{T}(s' \mid s, a) \left[\mathcal{R}(s, a, s') + \gamma V(s') \right] \tag{2.48}$$

3: Return optimal policy π^* with

$$\forall s \in S: \; \pi^*(s) \leftarrow \arg\max_{a \in A} \sum_{s' \in S} \mathcal{T}(s' \mid s, a) \left[\mathcal{R}(s, a, s') + \gamma V(s') \right] \tag{2.49}$$

Therefore, if π' did not change from π after policy improvement, we know that π' must be the optimal policy of the MDP, and policy iteration is complete.

The above version of policy iteration, which uses iterative policy evaluation, makes multiple complete update sweeps over the entire state space S. *Value iteration* is a DP algorithm that combines one sweep of iterative policy evaluation and policy improvement in a single update equation by using the Bellman optimality equation as an update operator:

$$V^{k+1}(s) \leftarrow \max_{a \in A} \sum_{s' \in S} \mathcal{T}(s' \mid s, a) \left[\mathcal{R}(s, a, s') + \gamma V^k(s') \right], \; \forall s \in S \tag{2.47}$$

Following a similar argument as for the Bellman operator, it can be shown that the Bellman optimality operator defined in Equation 2.47 is a γ-contraction mapping. Thus, repeated application of value iteration converges to the unique fixed point that is the optimal value function V^*. Algorithm 1 provides the complete pseudocode for the value iteration algorithm.

Returning to the Mars Rover example, if we run value iteration to convergence, we obtain the optimal state values $V^*(\textit{Start}) = 4.1$, $V^*(\textit{Site A}) = 6.2$, and $V^*(\textit{Site B}) = 10$. The optimal policy π^* chooses action *left* with probability 1 in the initial state and action *right* with probability 1 in states *Site A* and *Site B*.

2.6 Temporal-Difference Learning

Temporal-difference learning (TD) is a family of RL algorithms that learn value functions and optimal policies based on experiences from interactions with the environment. The experiences are generated by following the agent-environment interaction loop shown in Figure 2.2: in state s^t, sample an action $a^t \sim \pi(\cdot \mid s^t)$ from policy π, then observe reward $r^t = \mathcal{R}(s^t, a^t, s^{t+1})$ and the new state $s^{t+1} \sim \mathcal{T}(\cdot \mid s^t, a^t)$. Like DP algorithms, TD algorithms learn value functions based on the Bellman equations and bootstrapping — estimating the value of a

state or action using value estimates of other states/actions. However, unlike DP algorithms, TD algorithms do not require complete knowledge of the MDP, such as the reward function and state transition probabilities. Instead, TD algorithms perform the policy evaluation and policy improvement tasks based solely on the experiences collected while interacting with the environment.

TD algorithms use the following general update rule to learn action-value functions:

$$Q(s^t, a^t) \leftarrow Q(s^t, a^t) + \alpha \left[\mathcal{X} - Q(s^t, a^t) \right] \tag{2.50}$$

where $\mathcal{X}$ is the *update target*, and $\alpha \in (0, 1]$ is the *learning rate* (or step size). The update target $\mathcal{X}$ is constructed based on experience samples (s^t, a^t, r^t, s^{t+1}) collected from interactions with the environment. A wide range of options exist to specify the update target, and here we present two basic variants.

Recall the Bellman equation for the action-value function Q^π for policy π:

$$Q^\pi(s, a) = \sum_{s' \in S} \mathcal{T}(s' \mid s, a) \left[\mathcal{R}(s, a, s') + \gamma \sum_{a' \in A} \pi(a' \mid s') Q^\pi(s', a') \right] \tag{2.51}$$

Sarsa (Sutton and Barto 2018) is a TD algorithm that constructs an update target based on Equation 2.51 by replacing the summations over successor states s' and actions a' as well as the reward function $\mathcal{R}(s, a, s')$ with their corresponding elements from the experience tuple $(s^t, a^t, r^t, s^{t+1}, a^{t+1})$ (hence the name Sarsa):

$$\mathcal{X} = r^t + \gamma Q(s^{t+1}, a^{t+1}) \tag{2.52}$$

Here, the action a^{t+1} is sampled from the policy π in the successor state s^{t+1}, $a^{t+1} \sim \pi(\cdot \mid s^{t+1})$. The complete Sarsa update rule is specified as

$$Q(s^t, a^t) \leftarrow Q(s^t, a^t) + \alpha \left[r^t + \gamma Q(s^{t+1}, a^{t+1}) - Q(s^t, a^t) \right]. \tag{2.53}$$

If π stays fixed, then it can be shown under certain conditions that Sarsa will learn the action-value function $Q = Q^\pi$. The first condition is that all state-action combinations $(s, a) \in S \times A$ must be tried an infinite number of times during learning. The second condition is given by the "standard stochastic approximation conditions," which state that the learning rate α must be reduced over time in a way that satisfies the following conditions:

$$\forall s \in S, a \in A: \quad \sum_{k=1}^{\infty} \alpha_k(s, a) \rightarrow \infty \quad \text{and} \quad \sum_{k=1}^{\infty} \alpha_k(s, a)^2 < \infty \tag{2.54}$$

where $\alpha_k(s, a)$ denotes the learning rate used when applying the update rule in Equation 2.53 after the kth selection of action a in state s. The left-hand sum in Equation 2.54 ensures that the learning rate is large enough to overcome initial learning conditions, while the right-hand sum ensures that the sequence will converge at a certain rate. Therefore, $\alpha_k(s, a) = \frac{1}{k}$ satisfies these conditions on the

Algorithm 2 Sarsa for MDPs (with ϵ-greedy policies)

Initialize: $Q(s,a)=0$ for all $s \in S, a \in A$
Repeat for every episode:
Observe initial state s^0
With probability ϵ: choose random action $a^0 \in A$
Otherwise: choose action $a^0 \in \arg\max_a Q(s^0,a)$
for $t=0,1,2,...$ **do**
  Apply action a^t, observe reward r^t and next state s^{t+1}
  With probability ϵ: choose random action $a^{t+1} \in A$
  Otherwise: choose action $a^{t+1} \in \arg\max_a Q(s^{t+1},a)$
  $Q(s^t,a^t) \leftarrow Q(s^t,a^t) + \alpha \left[r^t + \gamma Q(s^{t+1},a^{t+1}) - Q(s^t,a^t) \right]$

learning rate, while a constant learning rate $\alpha_k(s,a)=c$ does not. Nonetheless, in practice it is common to use a constant learning rate since learning rates that meet the above conditions, while theoretically sound, can lead to slow learning.

In order for Sarsa to learn the optimal value function Q^* and optimal policy π^*, it must gradually modify the policy π in a way that brings it closer to the optimal policy. To achieve this, π can be made greedy with respect to the value estimates Q, similarly to the DP policy improvement task defined in Equation 2.39. However, making π fully greedy and deterministic would violate the first condition of trying all state-action combinations infinitely often. Thus, a common technique in TD algorithms is to instead use an *ϵ-greedy* policy that uses a parameter $\epsilon \in [0,1]$, defined as[6]

$$\pi(a \mid s) = \begin{cases} 1-\epsilon+\frac{\epsilon}{|A|} & \text{if } a \in \arg\max_{a' \in A} Q(s,a') \\ \frac{\epsilon}{|A|} & \text{otherwise} \end{cases} . \tag{2.55}$$

Thus, the ϵ-greedy policy chooses the greedy action with probability $1-\epsilon$, and with probability ϵ chooses a random other action. In this way, if $\epsilon > 0$, the requirement to try all state-action combinations an infinite number of times is ensured. Now, in order to learn the optimal policy π^*, we can gradually reduce the value of ϵ to 0 during learning, such that π will gradually converge to π^*. Pseudocode for Sarsa using ϵ-greedy policies is given in Algorithm 2.

6. Equation 2.55 assumes that there is a single greedy action with maximum value under Q. If multiple actions with maximum value exist for a given state, then the definition can be modified to distribute the probability mass $(1-\epsilon)$ among these actions.

Algorithm 3 Q-learning for MDPs (with ϵ-greedy policies)

1: Initialize: $Q(s,a) = 0$ for all $s \in S, a \in A$
2: Repeat for every episode:
3: **for** $t = 0, 1, 2, \ldots$ **do**
4: Observe current state s^t
5: With probability ϵ: choose random action $a^t \in A$
6: Otherwise: choose action $a^t \in \arg\max_a Q(s^t, a)$
7: Apply action a^t, observe reward r^t and next state s^{t+1}
8: $Q(s^t, a^t) \leftarrow Q(s^t, a^t) + \alpha \left[r^t + \gamma \max_{a'} Q(s^{t+1}, a') - Q(s^t, a^t) \right]$

While Sarsa is based on the Bellman equation for Q^π, one can also construct a TD update target based on the Bellman optimality equation for Q^*,

$$Q^*(s,a) = \sum_{s' \in S} \mathcal{T}(s' \mid s, a) \left[\mathcal{R}(s,a,s') + \gamma \max_{a' \in A} Q^*(s', a') \right]. \tag{2.56}$$

Q-learning (Watkins and Dayan 1992) is a TD algorithm that constructs an update target based on the above Bellman optimality equation, again by replacing the summation over states s' and the reward function $\mathcal{R}(s,a,s')$ with corresponding elements from the experience tuple (s^t, a^t, r^t, s^{t+1}):

$$\mathcal{X} = r^t + \gamma \max_{a' \in A} Q(s^{t+1}, a') \tag{2.57}$$

The complete Q-learning update rule is specified as

$$Q(s^t, a^t) \leftarrow Q(s^t, a^t) + \alpha \left[r^t + \gamma \max_{a' \in A} Q(s^{t+1}, a') - Q(s^t, a^t) \right]. \tag{2.58}$$

Pseudocode for Q-learning using ϵ-greedy policies is given in Algorithm 3.

Q-learning is guaranteed to converge to the optimal policy π^* under the same conditions as Sarsa (i.e., trying all state-action pairs infinitely often, and standard stochastic approximation conditions in Equation 2.54). However, in contrast to Sarsa, Q-learning does not require that the policy π used to interact with the environment be gradually made closer to the optimal policy π^*. Instead, Q-learning may use *any* policy to interact with the environment, so long as the convergence conditions are upheld. For this reason, Q-learning is called an *off-policy* TD algorithm while Sarsa is an *on-policy* TD algorithm, and this distinction has a number of implications which we will discuss in more detail in Chapter 8. In Section 2.7, we will use learning curves to evaluate and compare the performance of Q-learning and Sarsa on an RL problem.

2.7 Evaluation with Learning Curves

The standard approach to evaluate the performance of an RL algorithm on a learning problem is via *learning curves*. A learning curve shows the performance of the learned policy over increasing training time, where performance can be measured in terms of the learning objective (e.g., discounted returns) as well as other secondary metrics.

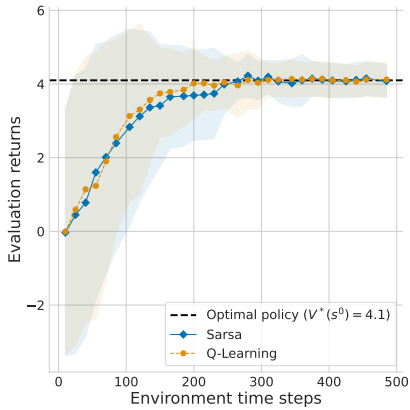
Figure 2.4 shows various learning curves for the Sarsa and Q-learning algorithms applied to the Mars Rover problem from Section 2.2 with discount factor $\gamma = 0.95$. In these figures, the x-axis shows the environment time steps across episodes, and the y-axis shows the average discounted *evaluation returns* achieved from the initial state, that is, $V^{\pi}(s^0)$.[7]

The term "evaluation return" means that the shown returns are for the greedy policy[8] with respect to the learned action values after T learning time steps. Thus, the plots answer the question: If we finish learning after T time steps and extract the greedy policy, what expected returns can we expect to achieve with this policy? The results shown here are averaged over one hundred independent training runs, each using a different random seed to determine the outcomes of random events (e.g., probabilistic action selections and state transitions). Specifically, each point on a line is produced by taking the greedy policy from each training run at that time, running one hundred independent episodes with the policy and averaging over the resulting returns, and finally averaging over the average returns corresponding to the one hundred training runs. The shaded area shows the standard deviation over the averaged returns from each training run. In Figure 2.4(b), we show the average episode lengths of the greedy policy instead of the average returns.

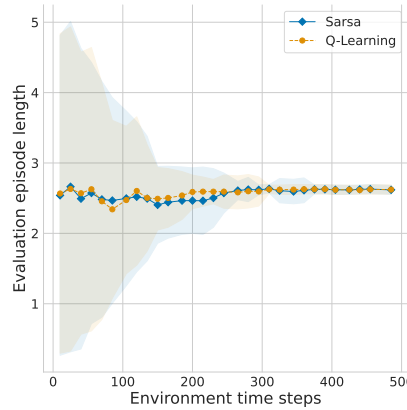
In this example, Sarsa and Q-learning both learn the optimal policy π^* which chooses action *left* in state *Start* and action *right* in states *Site A* and *Site B*. Moreover, the two algorithms basically have the same learning curves for the evaluation returns. Figures 2.4(c) and 2.4(d) (for Q-learning) show that the choice of the learning rate α and exploration rate ϵ can have an important impact on the learning. In this case, the "average" learning rate $\alpha_k(s,a) = \frac{1}{k}$ performs best for Q-learning, but for more complex MDPs with larger state and action sets, this choice of learning rate usually leads to much slower learning compared to appropriately chosen constant learning rates.

7. Recall that we use the term "return" as a shorthand for discounted return when the learning objective is to maximize discounted returns.

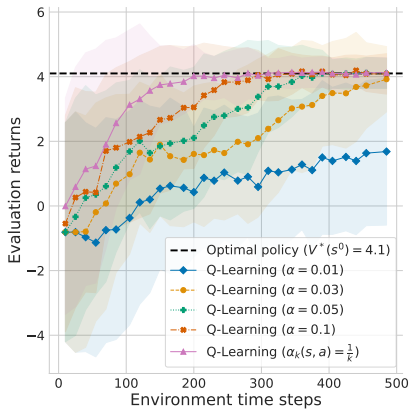
8. In policy gradient RL algorithms, discussed in Chapter 8, the algorithm learns a probabilistic policy for which there usually is no "greedy" version. In this case, the evaluation return just uses the current policy without modification.



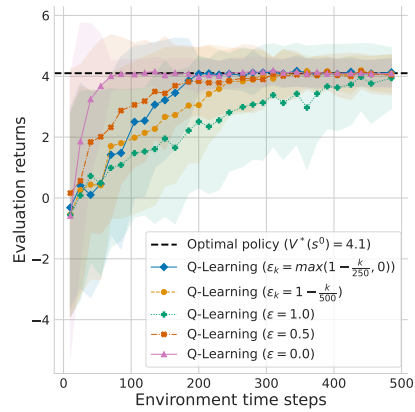
(a) Average evaluation returns for Sarsa and Q-learning



(b) Average episode length for Sarsa and Q-learning



(c) Q-learning with different learning rates α (ϵ linearly decayed from 1 to 0)



(d) Q-learning with different exploration rates ϵ ($\alpha = 0.1$)

Figure 2.4: Results when applying the Sarsa and Q-learning algorithms to the Mars Rover problem (Figure 2.3, page 23) with $\gamma = 0.95$, averaged over one hundred independent training runs. The shaded area shows the standard deviation over the averaged discounted returns from each training run. The dashed horizontal line marks the optimal value of the initial state ($V^*(s^0)$), computed via value iteration. In Figures 2.4(a) and 2.4(b), the algorithms use a scheduled learning rate of $\alpha_k(s,a) = \frac{1}{k}$ where k is the number of times state s has been visited and action a was applied. The exploration rate is linearly decayed from $\epsilon = 1$ to $\epsilon = 0$ over time steps $t = 0, ..., 500$ (i.e., $\epsilon^t = 1 - t\frac{1}{500}$).

Note that the x-axis of the plots show the cumulative training time steps *across* episodes. This is not to be confused with the time steps t *within* episodes. The reason that we show cumulative time steps instead of the number of completed episodes on the x-axis is that the latter can potentially skew the comparison of algorithms. For instance, suppose we show training curves for algorithms A and B with number of episodes on the x-axis. Suppose both algorithms eventually reach the same performance with their learned policies, but algorithm A learns much faster than algorithm B (i.e., the curve for A rises more quickly). In this case, it could be that in the early episodes, algorithm A explored many more actions (i.e., time steps) than algorithm B before finishing the episode, which results in a larger number of collected experiences (or training data) for algorithm A. Thus, although both algorithms completed the same number of episodes, algorithm A will have done more learning updates (assuming updates are done after each time step, as in the TD methods presented in this chapter), which would explain the seemingly faster growth in the learning curve. If we instead showed the learning curve for both algorithms with cumulative time steps across episodes, then the learning curve for A might not grow faster than the curve for B.

Recall that at the beginning of this chapter (Figure 2.1, page 20), we defined an RL problem as the combination of a decision process model (e.g., MDP) and a learning objective for the policy (e.g., maximizing the expected discounted return in each state for a specific discount factor). When evaluating a learned policy, we want to evaluate its ability to achieve this learning objective. For example, our Mars Rover problem specifies a discounted return objective with the discount factor $\gamma = 0.95$. Therefore, our learning curves in Figure 2.4(a) show exactly this objective on the y-axis.

However, in some cases, it can be useful to show *undiscounted returns* (i.e., $\gamma = 1$) for a learned policy, even if the RL problem specified a discounted return objective with a specific discount factor $\gamma < 1$. The reason is that undiscounted returns can sometimes be easier to interpret than discounted returns. For example, suppose we want to learn an optimal policy for a video game in which the agent controls a space ship and receives a +1 score (reward) for each destroyed enemy. If in an episode the agent destroys ten enemies at various points in time, the undiscounted return will be ten but the discounted return will be less than ten, making it more difficult to understand how many enemies the agent destroyed. More generally, when analyzing a learned policy, it is often useful to show additional metrics besides the returns, such as win rates in competitive games, or episode lengths as we did in Figure 2.4(b).

When showing undiscounted returns and additional metrics such as episode lengths, it is important to keep in mind that the evaluated policy was not actually

trained to maximize these objectives — it was trained to maximize the expected discounted return for a specific discount factor. In general, two RL problems that differ only in their discount factor but are otherwise identical (i.e., same MDP) may have different optimal policies. In our Mars Rover example, a discounted return objective with discount factor $\gamma = 0.95$ leads to an optimal policy that chooses action *left* in the initial state, achieving a value of $V^*(Start) = 4.1$. In contrast, a discount factor of $\gamma = 0.5$ leads to an optimal policy that chooses action *right* in the initial state, achieving a value of $V^*(Start) = 0$. Both policies are *optimal*, but they are for two different learning problems that use different discount factors. Both learning problems are valid, and the choice of γ is part of designing the learning problem.

2.8 Equivalence of $\mathcal{R}(s, a, s')$ and $\mathcal{R}(s, a)$

Our definition of MDPs uses the most general definition of reward functions, $\mathcal{R}(s^t, a^t, s^{t+1})$, by conditioning on the current state s^t and action a^t as well as the successor state s^{t+1}. Another definition for reward functions commonly used in the RL literature is to condition only on s^t and a^t, that is, $\mathcal{R}(s^t, a^t)$. This may be a point of confusion for newcomers to RL. However, these two definitions are in fact equivalent, in that for any MDP using $\mathcal{R}(s^t, a^t, s^{t+1})$ we can construct an MDP using $\mathcal{R}(s^t, a^t)$ (all other components are identical to the original MDP) such that a given policy π produces the same expected returns in both MDPs.

Recall the Bellman equation for the state-value function V^π that we have been using so far for an MDP with reward function $\mathcal{R}(s, a, s')$:

$$V^\pi(s) = \sum_{a \in A} \pi(a \mid s) \sum_{s' \in S} \mathcal{T}(s' \mid s, a) \left[\mathcal{R}(s, a, s') + \gamma V^\pi(s') \right] \tag{2.59}$$

Assuming that $\mathcal{R}(s, a, s')$ depends only on s, a, we can rewrite the above equation as

$$V^\pi(s) = \sum_{a \in A} \pi(a \mid s) \sum_{s' \in S} \mathcal{T}(s' \mid s, a) \left[\mathcal{R}(s, a) + \gamma V^\pi(s') \right] \tag{2.60}$$

$$= \sum_{a \in A} \pi(a \mid s) \left[\sum_{s' \in S} \mathcal{T}(s' \mid s, a) \mathcal{R}(s, a) + \sum_{s' \in S} \mathcal{T}(s' \mid s, a) \gamma V^\pi(s') \right] \tag{2.61}$$

$$= \sum_{a \in A} \pi(a \mid s) \left[\mathcal{R}(s, a) + \gamma \sum_{s' \in S} \mathcal{T}(s' \mid s, a) V^\pi(s') \right]. \tag{2.62}$$

This is the simplified Bellman equation for MDPs using a reward function $\mathcal{R}(s, a)$. Going the reverse direction from $\mathcal{R}(s, a)$ to $\mathcal{R}(s, a, s')$, we know that by

defining $\mathcal{R}(s,a)$ to be the expected reward under the state transition probabilities

$$\mathcal{R}(s,a) = \sum_{s' \in S} \mathcal{T}(s' \mid s,a)\mathcal{R}(s,a,s') \tag{2.63}$$

and plugging this into Equation 2.62, we recover the original Bellman equation that uses $\mathcal{R}(s,a,s')$, given in Equation 2.59. Analogous transformations can be done for the Bellman equation of the action-value function Q^π, as well as the Bellman optimality equations for V^*/Q^*. Therefore, one may use either $\mathcal{R}(s,a,s')$ or $\mathcal{R}(s,a)$.

In this book, we use $\mathcal{R}(s,a,s')$ for the following pedagogical reasons:

1. Using $\mathcal{R}(s,a,s')$ can be more convenient when specifying examples of MDPs, since it is useful to show the differences in reward for multiple possible outcomes of an action. In our Mars Rover MDP shown in Figure 2.3, when using $\mathcal{R}(s,a)$ we would have to show the expected reward (Equation 2.63) on the transition arrow for action *right* from the initial state *Start* (rather than showing the different rewards for the two possible outcomes), which would be less intuitive to read.
2. The Bellman equations when using $\mathcal{R}(s,a,s')$ show a helpful visual congruence with the update targets used in TD algorithms. For example, the Q-learning update target, $r^t + \gamma \max_{a'} Q(s^{t+1}, a')$, appears in a corresponding form in the Bellman optimality equation (in the [] brackets):

$$Q^*(s,a) = \sum_{s' \in S} \mathcal{T}(s' \mid s,a) \left[\mathcal{R}(s,a,s') + \gamma \max_{a' \in A} Q^*(s',a') \right] \tag{2.64}$$

   And similarly for the Sarsa update target and the Bellman equation for Q^π (see Section 2.6).

Finally, we mention that most of the original MARL literature presented in Chapter 6 in fact defined reward functions as $\mathcal{R}(s,a)$. To stay consistent in our notation, we will continue to use $\mathcal{R}(s,a,s')$ in the remainder of this book, noting that equivalent transformations from $\mathcal{R}(s,a)$ to $\mathcal{R}(s,a,s')$ always exist.

2.9 Summary

This chapter has provided a concise introduction to the basic concepts of single-agent RL. The most important concepts are the following:

- The *Markov decision process* (MDP) is the standard model to define the environment in which the agent chooses actions over a number of time steps. It defines the possible states of the environment, the actions available to the agent, how the environment state changes in response to the different actions,

and the rewards received by the agent. The agent uses a policy that assigns probabilities to the available actions in each state.

- A learning problem in RL is defined by the combination of a decision process model (e.g., MDP) and a learning objective. The most common learning objective used in single-agent RL is to find a decision policy for the agent that maximizes the *expected discounted return*, defined as the sum of rewards received over time and weighted by a discount factor.
- The *Markov property* in MDPs means that the future states and rewards are independent of past states and actions, given the current state and action. This property allows us to define recursive value functions for policies, known as *Bellman equations*, that return the "value" of a policy in each possible state of the environment. The value is the expected return when starting in the given state and following the policy to select actions. Value functions can also be defined for state-action pairs, where a given action is first chosen and subsequent actions are chosen by the policy.
- *Dynamic programming* (DP) and *temporal-difference learning* (TD) are two families of algorithms that can learn optimal policies in MDPs. DP algorithms require complete knowledge of the MDP specification and use procedures such as value iteration to learn optimal value functions. TD algorithms build on DP theory but do not need complete knowledge of the MDP, instead learning optimal policies by repeatedly exploring actions in the environment and observing the outcomes.
- RL algorithms are evaluated via *learning curves* that show improvement in expected returns over an increasing number of agent-environment interactions during learning.

The following chapters in this part of the book will build on the above concepts in multiple ways. First, Chapter 3 will extend the MDP model by introducing game models in which multiple agents interact in a shared environment. Chapter 4 will use the notion of discounted returns to define a range of solution concepts for multi-agent games. Finally, Chapters 5 and 6 will introduce several families of MARL algorithms that build on and extend the DP and TD algorithms introduced in this chapter.